



## LeetCode 746: Min Cost Climbing Stairs

### 1. Problem Title & Link

**Title:** LeetCode 746: Min Cost Climbing Stairs

**Link:** <https://leetcode.com/problems/min-cost-climbing-stairs/>

### 2. Problem Statement (Short Summary)

You are given:

$\text{cost}[i]$

where:

$\text{cost}[i]$

=

cost of stepping on stair  $i$

You can climb:

1 step

or

2 steps

at a time.

You can start from:

step 0

or

step 1

Return:

Minimum cost

to reach the top.

### 3. Examples (Input → Output)

#### Example 1

Input:

$\text{cost} = [10, 15, 20]$

Output:

15

Explanation:

Start at step 1

Pay 15

Jump to top

**Example 2**

Input:

cost = [1,100,1,1,1,100,1,1,100,1]

Output:

6

**4. Constraints**

- $2 \leq \text{cost.length} \leq 1000$
- $0 \leq \text{cost}[i] \leq 999$

**5. Core Concept (Pattern / Topic)****Dynamic Programming**

Related Topics:

- 1D DP
- Fibonacci Pattern
- Optimization

**6. Thought Process (Step-by-Step Explanation)****Understanding the Problem**

Suppose:

cost = [10,15,20]

You can:

**Path 1**

10 → 20

Cost:

30

**Path 2**

15

Cost:

15

Minimum:

15

**Key Observation**

To reach stair:



i

you can come from:

i-1

or

i-2

Choose cheaper path.

### DP Definition

Let:

$dp[i]$

be:

Minimum cost

to reach stair i

### Recurrence

To reach stair:

i

Either:

Come from i-1

Cost:

$dp[i-1] + cost[i-1]$

or

Come from i-2

Cost:

$dp[i-2] + cost[i-2]$

Take minimum.

Formula:

$dp[i] =$

$\min($

$dp[i-1] + cost[i-1],$

$dp[i-2] + cost[i-2]$

$)$

### 7. Visual / Intuition Diagram (ASCII Diagram)

Input:

10    15    20

0    1    2

Top:



```
      TOP
     /  \
    20   15
Choose:
15
Minimum cost.
```

## 8. Pseudocode

```
dp[0] = 0
dp[1] = 0

for i from 2 to n:

    dp[i] =
    min(
        dp[i-1] + cost[i-1],
        dp[i-2] + cost[i-2]
    )

return dp[n]
```

## 9. Code Implementation

### ✓ Python

```
class Solution:

    def minCostClimbingStairs(self, cost):

        n = len(cost)

        dp = [0] * (n + 1)

        for i in range(2, n + 1):

            dp[i] = min(
                dp[i - 1] + cost[i - 1],
                dp[i - 2] + cost[i - 2]
            )

        return dp[n]
```

### ✓ Space Optimized Python

```
class Solution:

    def minCostClimbingStairs(self, cost):
```



```
prev2 = 0
prev1 = 0

for i in range(2, len(cost) + 1):

    current = min(
        prev1 + cost[i - 1],
        prev2 + cost[i - 2]
    )

    prev2 = prev1
    prev1 = current

return prev1
```

### ✓ Java

```
class Solution {

    public int minCostClimbingStairs(
        int[] cost) {

        int n = cost.length;

        int[] dp = new int[n + 1];

        for (int i = 2;
            i <= n;
            i++) {

            dp[i] = Math.min(
                dp[i - 1]
                + cost[i - 1],

                dp[i - 2]
                + cost[i - 2]
            );

        }

        return dp[n];

    }

}
```

### ✓ Space Optimized Java

```
class Solution {
```



```
public int minCostClimbingStairs(  
    int[] cost) {  
  
    int prev2 = 0;  
    int prev1 = 0;  
  
    for (int i = 2;  
        i <= cost.length;  
        i++) {  
  
        int current = Math.min(  
            prev1 + cost[i - 1],  
            prev2 + cost[i - 2]  
        );  
  
        prev2 = prev1;  
        prev1 = current;  
    }  
  
    return prev1;  
}
```

## 10. Time & Space Complexity

### DP Array

#### Time

$O(n)$

#### Space

$O(n)$

### Space Optimized

#### Time

$O(n)$

#### Space

$O(1)$

## 11. Common Mistakes / Edge Cases

### Mistake 1

Using:

`cost[i]`

instead of:

`cost[i-1]`

in recurrence.

**Mistake 2**

Forgetting that top is:

n

not:

n-1

**Mistake 3**

Initializing DP incorrectly.

Correct:

dp[0] = 0

dp[1] = 0

**12. Detailed Dry Run**

```
Input:
cost = [10,15,20]
n:
3
DP:
[0,0,0,0]

i = 2
dp[2]

=
min(
0+15,
0+10
)

=
10
DP:
[0,0,10,0]

i = 3
dp[3]

=
min(
10+20,
0+15
)

=
```



```
15
DP:
[0, 0, 10, 15]
Answer:
15
```

### 13. Common Use Cases

#### Cost Optimization

Find cheapest path.

#### Resource Planning

Minimum expenditure route.

#### Energy Saving

Choose least costly jumps.

#### Dynamic Routing

Minimum weighted transitions.

### 14. Common Traps

- Wrong indexing
- Returning  $dp[n-1]$
- Missing top stair concept
- Forgetting space optimization

### 15. Builds To (Related Problems)

1. LC 70 — Climbing Stairs
2. LC 198 — House Robber
3. LC 746 — Min Cost Climbing Stairs
4. LC 740 — Delete and Earn
5. LC 1137 — N-th Tribonacci Number

### 16. Alternate Approaches

| Approach    | Time     | Space  | Notes |
|-------------|----------|--------|-------|
| Recursion   | $O(2^n)$ | $O(n)$ | TLE   |
| Memoization | $O(n)$   | $O(n)$ | Good  |





|                    |        |        |         |
|--------------------|--------|--------|---------|
| Tabulation         | $O(n)$ | $O(n)$ | Better  |
| Space Optimized DP | $O(n)$ | $O(1)$ | Optimal |

### 17. Why This Solution Works

To reach any stair, only the previous two stairs matter. The minimum cost to reach stair  $i$  is therefore the minimum of coming from  $i-1$  or  $i-2$ , making it a classic Dynamic Programming problem.

### 18. Variations / Follow-Up Questions

#### Follow-Up 1

Can we climb 1, 2, or 3 stairs?

Modify recurrence to consider three previous states.

#### Follow-Up 2

Maximum reward instead of minimum cost?

Replace min with max.

#### Follow-Up 3

Variable jump lengths?

Use generalized DP.